BLOQUE IV - TEMA 3

SISTEMAS Y COMUNICACIONES

Administración de servidores de correo electrónico sus protocolos. Administración de contenedores y microservicios.

Supera. by thePower

© Reservados todos los derechos de esta edición para POWEREDUCATION S.L., 2024.

ÍNDICE

1. ADMINISTRACIÓN DE SERVIDORES DE CORREO. SUS PROTOCOLOS. 5

1.1 Servidores de correo electrónico 5

1.1.1 MTA y MUA. Otros agentes de correo. 5

1.1.2 Intercambio de correos 6

1.1.3 Administrador de correo electrónico. Funciones 7

1.2 Protocolos de correo electrónico 8

1.2.1 SMTP (Simple Mail Transfer Protocol) 8

1.2.2 POP3 (Post Office Protocol v3) 9

1.2.3 IMAP (Internet Message Access Protocol) 10

1.2.4 Exchange ActiveSync 11

1.2.5 Comparativa entre IMAP y POP3 11

2. ADMINISTRACIÓN DE CONTENEDORES Y MICROSERVICIOS 13

2.1 Arquitectura de Microservicios 13

2.1.1 Características de los microservicios 14

2.1.2 Ventajas e inconvenientes de los microservicios 14

2.1.3 Comunicación entre microservicios 14

2.1.4 Integración entre microservicios: Orquestación y Coreografía 16

2.2 Contenedores 17

2.2.1 Arquitectura de contenedores 17

2.2.2 Características. Ventajas e inconvenientes 19

2.3 Plataformas de contenedores y microservicios 19

2.3.1 Docker 19

2.3.2 Kubernetes 19

2.3.3 OpenShift 21

2.3.4 Rancher 22

2.3.5 Apache Mesos 22

2.3.6 Otras plataformas relevantes 23

Significado de los iconos que aparecen dentro de los TEMAS:



Examen


Importante


Recordatorio


Atención

© Reservados todos los derechos de esta edición para POWEREDUCATION S.L., 2025.

1. ADMINISTRACIÓN DE SERVIDORES DE CORREO. SUS PROTOCOLOS.

1.1 SERVIDORES DE CORREO ELECTRÓNICO

Un servidor de correo electrónico es un sistema informático encargado de enviar, recibir, almacenar y reenviar mensajes de correo electrónico entre usuarios. Estos servidores permiten que los mensajes lleguen desde el remitente hasta el destinatario a través de una red, normalmente Internet, o una red corporativa. Según el CCN, deben ubicarse en la zona desmitalirizada (DMZ).

1.1.1 MTA y MUA. Otros agentes de correo.

En la arquitectura del correo electrónico intervienen distintos componentes. Dos de los más importantes son:

- **MTA (Mail Transfer Agent)**: es el agente de transferencia de mensajes, que reside en el servidor de correo. Su función es enviar correos electrónicos desde el servidor del remitente hacia el del destinatario, o bien actuar como intermediario entre MTA. Ejemplos de MTA son *Postfix*, *Sendmail*, *Exim* o *Microsoft Exchange Transport*. Los MTA escuchan en los puertos 25 y 587.

- **MUA (Mail User Agent)**: es el agente de usuario de correo. Es el programa que usa el usuario final para redactar, enviar, leer y organizar correos. Los MUAs pueden ser aplicaciones de escritorio (como Outlook o Thunderbird), aplicaciones móviles (como Gmail en Android o iOS) o aplicaciones web.

- **MDA (Mail Delivery Agent)**: agente de entrega de correo. Son programas que pueden utilizar los MTA para entregar el correo electrónico al buzón del usuario.

- **MSA (Mail Submission Agent)**: agente de registro de correo. En servicios con muchos usuarios y carga, mejora la eficiencia del sistema, ya que facilita la división del trabajo en los MTA, asumiendo ciertas tareas como el direccionamiento.

- **MAA (Mail Acess Agent)**: agente de acceso al correo. Se utiliza para recuperar el buzón de mensajes de un servidor de correo. IMAP y POP son ejemplos de MAA.

© Reservados todos los derechos de esta edición para POWEREDUCATION S.L., 2025

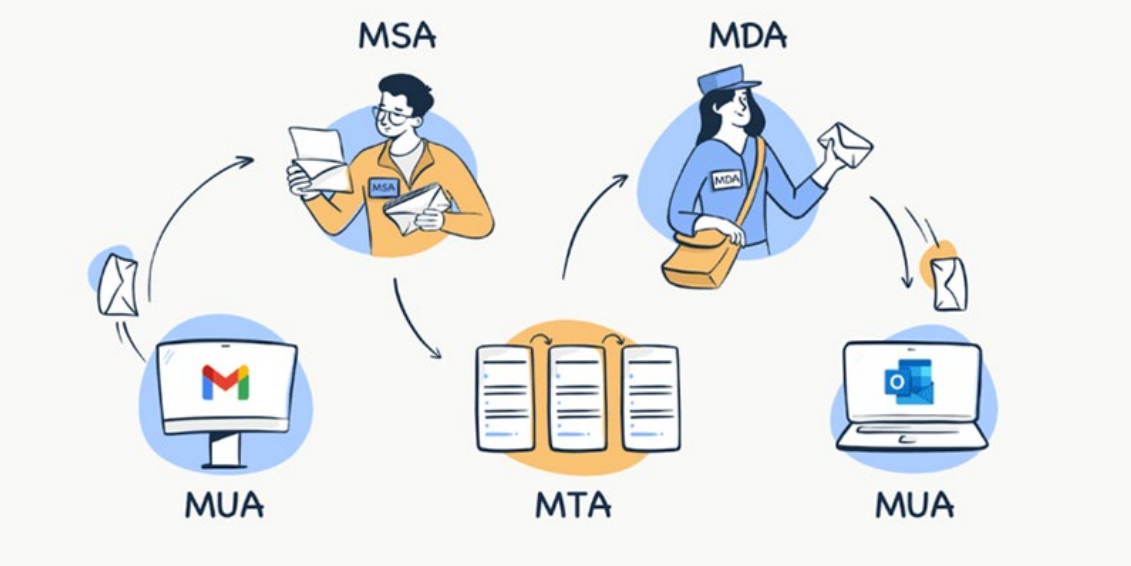


Ilustración 1. Ejemplo de intercambio de correos entre MUA

1.1.2 Intercambio de correos

El intercambio de correos electrónicos se realiza siguiendo una serie de pasos definidos por protocolos estandarizados:

1. El usuario redacta y envía un correo desde su cliente (MUA).
2. El MUA transmite el mensaje al servidor de salida (MTA) usando SMTP.
3. El MTA del remitente busca, a través del DNS, el servidor MTA del dominio del destinatario.
4. El mensaje se transfiere al servidor MTA del destinatario.
5. El MTA del destinatario entrega el mensaje en el buzón del usuario (usando el MDA).
6. El destinatario accede a su correo mediante su MUA, conectándose al servidor de correo mediante IMAP o POP3.

Este proceso suele ser transparente para el usuario, pero debe ser entendido por los administradores del sistema para poder diagnosticar problemas y configurar adecuadamente los servicios, como el DNS.

© Reservados todos los derechos de esta edición para POWEREDUCATION S.L., 2025

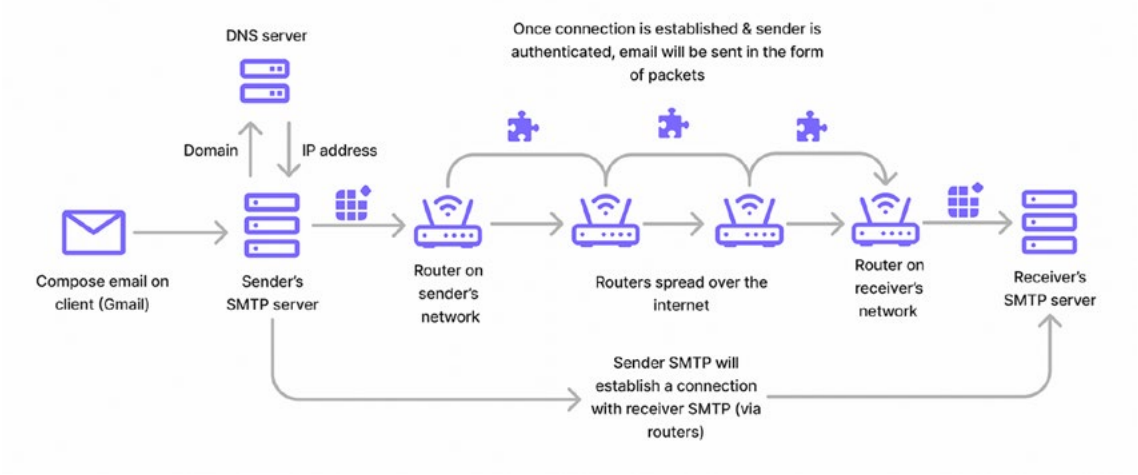


Ilustración 2. Proceso de intercambio de correos.

Fuente: https://medium.com/building-mailmodo/how-does-email-work-a-dummies-guide-bd1ef894797e

1.1.3 Administrador de correo electrónico. Funciones

El administrador de correo electrónico es el responsable de gestionar y mantener en funcionamiento el sistema de correo de una organización. Sus principales funciones incluyen:

- **Configuración del servidor de correo**: instalación y ajuste de parámetros del MTA (como Postfix o Exchange), incluyendo nombre del dominio, alias, límites de tamaño, etc.
- **Gestión de usuarios y buzones**: alta, baja y modificación de cuentas de correo, gestión de cuotas y políticas de almacenamiento.
- **Seguridad del correo electrónico**: implementación de medidas contra el spam, suplantación de identidad (phishing), malware y otras amenazas. Configuración de protocolos como SPF, DKIM, DMARC y cifrado TLS.
- **Supervisión y monitorización**: control del estado del servidor, uso de recursos, entrega de mensajes, errores de entrega y análisis de registros (logs).
- **Backup y recuperación**: establecimiento de políticas de copia de seguridad de buzones y restauración ante pérdida o corrupción de datos.
- **Soporte a usuarios**: resolución de incidencias relacionadas con el acceso al correo, configuración de clientes, reglas de filtrado, etc.
- **Actualizaciones y mantenimiento**: aplicación de parches de seguridad y actualizaciones del software del servidor de correo.

© Reservados todos los derechos de esta edición para POWEREDUCATION S.L., 2025

1.2 PROTOCOLOS DE CORREO ELECTRÓNICO

Los protocolos de correo electrónico definen cómo se envían, reciben y consultan los mensajes entre los servidores y los usuarios. Los más importantes son:

- **SMTP** (Simple Mail Transfer Protocol), para el envío de mensajes.
- **POP3** (Post Office Protocol versión 3) e **IMAP** (Internet Message Access Protocol), para la recepción y gestión de mensajes por parte del usuario.
- **Exchange ActiveSync**, un protocolo propietario de Microsoft orientado a la sincronización de datos en dispositivos móviles.

1.2.1 SMTP (Simple Mail Transfer Protocol)



SMTP es el protocolo estándar para el envío de correos electrónicos, basado en el modelo cliente-servidor. Se utiliza entre clientes y servidores, y entre servidores, para transportar mensajes. Trabaja típicamente sobre el puerto TCP 25 (o el 587 para envío autenticado). A diferencia de POP3 o IMAP, SMTP no facilita la recuperación de mensajes de correo electrónico desde el servidor.

SMTP está definido en las RFC 821, 822 y 5321.

Órdenes básicas de SMTP

Las órdenes SMTP son comandos en texto plano que se utilizan para establecer la comunicación y transferir el mensaje.

Orden	Descripción
HELO	Identifica al cliente frente al servidor. Se usa al inicio de la conexión.
MAIL FROM	Especifica la dirección del remitente del mensaje.
RCPT TO	Indica la dirección del destinatario. Puede repetirse para varios.
DATA	Indica el comienzo del mensaje. Finaliza con una línea un punto.
QUIT	Termina la sesión SMTP.
SEND	Solicita la entrega del mensaje directamente al buzón del destinatario.
SAML	Similar a SEND, pero se entrega el mensaje al buzón y se reenvía.
SOML	Envía el mensaje al buzón o lo reenvía, según disponibilidad.
VRFY	Verifica si existe una dirección de correo en el servidor.
HELP	Solicita ayuda al servidor sobre los comandos disponibles.
TURN	Invierte roles cliente/servidor (obsoleto por razones de seguridad).

© Reservados todos los derechos de esta edición para POWEREDUCATION S.L., 2025

Códigos de respuesta SMTP

Son códigos de respuesta del servidor SMTP, para indicar la categoría de la misma:

Código	Significado general	Descripción
2XX	Éxito	La operación se ha completado con éxito.
3XX	Acción pendiente	La orden ha sido aceptada, pero se requiere más información o acción por parte del cliente.
4XX	Error temporal	El comando no se ha completado, pero puede volver a intentarse.
5XX	Error permanente	La operación ha fallado de forma definitiva.

Para evitar problemas en la falsificación de direcciones de envío, se utiliza el protocolo **SPF (Sender Policy Framework)** que identifica a los servidores SMPT autorizados, a través de sus DNS.

1.2.2 POP3 (Post Office Protocol v3)



POP3 es un protocolo cliente- servidor de recepción de correos, especificado en la RFC 1939. Su funcionamiento se basa en **descargar los mensajes del servidor al cliente y eliminarlos**, aunque puede configurarse para conservar copias en el servidor. Utiliza el puerto TCP 110 (ó 995 si se usa cifrado SSL/TLS).

Órdenes básicas de POP3

Orden	Descripción
USER	Indica el nombre de usuario para autenticarse.
PASS	Proporciona la contraseña del servidor.
STAT	Muestra el número de mensajes y su tamaño total.
LIST	Lista los mensajes y su tamaño, uno por uno.
RETR	Recupera un mensaje específico.
TOP	Recupera las cabeceras y las primeras líneas de un mensaje.
DELE	Marca un mensaje para su eliminación.
RSET	Cancela las eliminaciones marcadas durante la sesión.
UIDL	Devuelve un identificador único para cada mensaje.
QUIT	Termina la sesión y elimina los mensajes marcados para borrar.

© Reservados todos los derechos de esta edición para POWEREDUCATION S.L., 2025

1.2.3 IMAP (Internet Message Access Protocol)

EXAMEN

IMAP es un protocolo de cliente de correo, que permite acceder a los mensajes directamente en el servidor sin necesidad de descargarlos. A diferencia de POP3, **mantiene los mensajes en el servidor y permite sincronización entre múltiples dispositivos**. Funciona típicamente en el puerto TCP 143 (o 993 si se usa cifrado SSL/TLS). Su versión actual es la 4.

Ventajas principales de IMAP:

- Sincronización en tiempo real con múltiples dispositivos.
- Organización de mensajes en carpetas en el servidor.
- Posibilidad de buscar mensajes en el servidor.
- Permite descargar solo partes del mensaje, como cabeceras o adjuntos concretos.

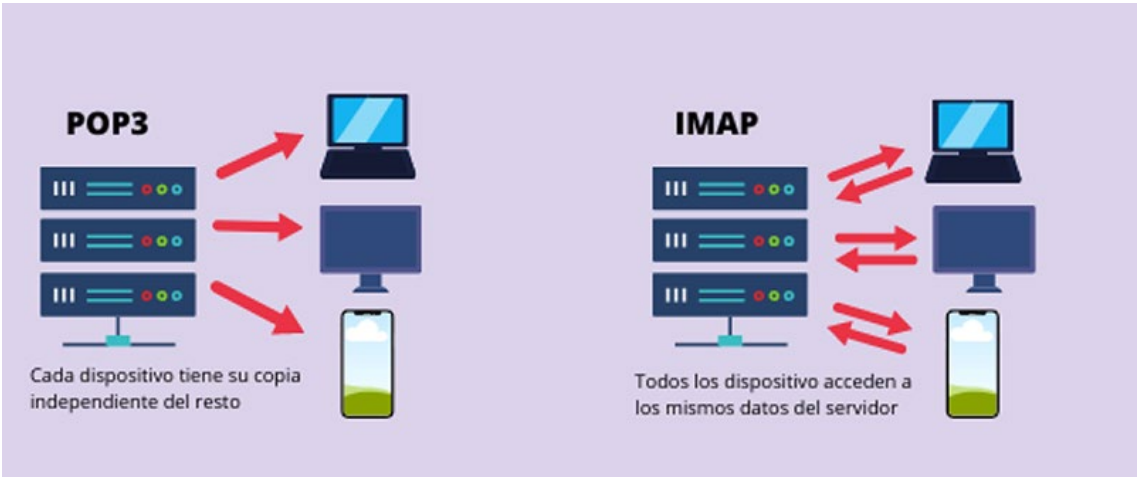


Ilustración 3. Diferencias entre POP3 e IMAP.

Fuente: https://ayuda.newweb.cl/diferencias-entre-pop3-e-imap/

© Reservados todos los derechos de esta edición para POWEREDUCATION S.L., 2025

1.2.4 Exchange ActiveSync



Exchange ActiveSync (EAS) es un **protocolo propietario de Microsoft**, basado en HTTP y XML, diseñado para la sincronización de Correos electrónicos, Calendarios, Contactos o Tareas y notas.

Está optimizado para dispositivos móviles y se utiliza principalmente en entornos corporativos con servidores Microsoft Exchange. No permite buzones compartidos ni acceso delegado. Sí que facilita el borrado remoto de un dispositivo (por ejemplo, si se pierde o roba). También facilita la utilización de cifrado SSL entre dispositivo móvil y servidor Exchange.

Ventajas frente a IMAP/POP3:

- Integra otros elementos (calendarios, tareas).
- Sincroniza cambios rápidamente en ambos sentidos.
- Mejor integración con dispositivos móviles modernos y Outlook.

1.2.5 Comparativa entre IMAP y POP3

Característica	IMAP (Internet Message Access Protocol)	POP3 (Post Office Protocol versión 3)
Modo de funcionamiento	Acceso y gestión del correo directamente en el servidor.	Descarga los correos del servidor al dispositivo local.
Almacenamiento	Los mensajes permanecen en el servidor (salvo configuración contraria).	Los mensajes se eliminan del servidor tras ser descargados (por defecto).
Sincronización	Sí. Permite acceder al mismo buzón desde varios dispositivos.	No. Solo se puede acceder desde un único dispositivo sin conflictos.
Gestión de carpetas	Permite crear, renombrar y eliminar carpetas en el servidor.	No permite gestionar carpetas, solo descarga la bandeja de entrada.
Acceso sin conexión	Se puede trabajar parcialmente sin conexión si se descarga el contenido.	Sí. Una vez descargado, el correo está disponible sin conexión.
Uso del almacenamiento	Requiere espacio en el servidor.	Requiere espacio en el dispositivo del usuario.
Complejidad	Más complejo de implementar, pero con mayores funcionalidades.	Más simple, orientado a usos básicos.
Consumo de datos	Mayor uso de datos al sincronizar constantemente.	Menor uso de datos al descargar los mensajes una sola vez.

© Reservados todos los derechos de esta edición para POWEREDUCATION S.L., 2025

Protocolo	Ventajas	Desventajas
IMAP	· Permite acceso desde múltiples dispositivos con sincronización en tiempo real. · Los mensajes se almacenan en el servidor y se mantienen sincronizados. · Gestión avanzada de carpetas y organización de correos. · Ideal para entornos colaborativos o profesionales.	· Requiere conexión a Internet para acceder a todos los mensajes. · Mayor uso de espacio en el servidor. · Mayor consumo de datos al sincronizar constantemente. · Puede ser más complejo de configurar y mantener.
POP3	· Permite acceder a los mensajes sin conexión, una vez descargados. · Ahorra espacio en el servidor, ya que los mensajes se descargan localmente. · Sencillo de configurar y mantener. · Menor uso de ancho de banda, útil con conexiones lentas.	· No permite sincronización entre dispositivos. · Los mensajes se eliminan del servidor por defecto tras la descarga. · No permite organizar los mensajes en carpetas en el servidor. · Difícil gestión si se accede al correo desde más de un dispositivo.

© Reservados todos los derechos de esta edición para POWEREDUCATION S.L., 2025

2. ADMINISTRACIÓN DE CONTENEDORES Y MICROSERVICIOS

2.1 ARQUITECTURA DE MICROSERVICIOS

La arquitectura de microservicios es un enfoque moderno para el diseño y desarrollo de aplicaciones. Consiste en dividir una aplicación en **módulos pequeños, independientes y especializados**, llamados *microservicios*. Cada uno de ellos cumple una función concreta del sistema y se ejecuta de forma autónoma.

A diferencia de las aplicaciones **monolíticas**, en las que todo el código está integrado en un único bloque, en los microservicios **cada módulo puede desarrollarse, desplegarse y escalarse de forma independiente**. Esta estrategia permite una mayor flexibilidad, agilidad y robustez en sistemas complejos, aportando además escalabilidad horizontal.

Ejemplo sencillo: una aplicación web de comercio electrónico puede tener los siguientes microservicios: microservicio de productos, microservicio de usuarios, microservicio de pagos y microservicio de envíos. Cada uno de ellos puede desarrollarse por un equipo distinto, con diferentes tecnologías, y escalarse según la demanda (por ejemplo, el de pagos podría requerir más capacidad en épocas de promociones).

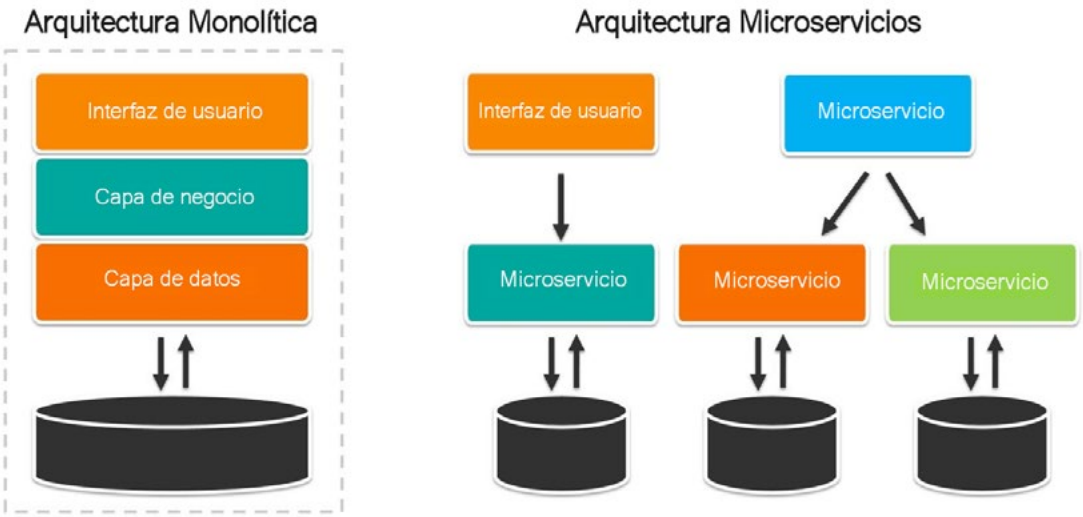


Ilustración 4. Diferencias entre arquitectura tradicional y microservicios. Fuente: https://blog.feedback-it.com/2020/10/13/microservicios-que-son-pros-contras-y-companias-que-la-utilizan/

© Reservados todos los derechos de esta edición para POWEREDUCATION S.L., 2025

2.1.1 Características de los microservicios

Los microservicios presentan las siguientes características clave:

- **Independencia funcional**: cada microservicio tiene una única funcionalidad.
- **Desarrollo y actualización desacoplados**: los equipos pueden trabajar de forma paralela en distintos servicios, sin interferencias. Facilita evolucionar la aplicación.
- **Escalado selectivo**: los servicios más exigentes pueden escalarse sin necesidad de duplicar toda la aplicación.
- **Despliegue autónomo**: se puede publicar una nueva versión de un servicio sin afectar al resto.
- **Resistencia a fallos**: si un microservicio falla, el resto del sistema puede seguir funcionando.
- **Comunicación a través de APIs**: generalmente mediante HTTP/REST, gRPC o eventos.
- **Persistencia propia**: cada servicio puede tener su propia base de datos o sistema de almacenamiento.

2.1.2 Ventajas e inconvenientes de los microservicios

En la siguiente tabla se recogen las ventajas e inconvenientes de los microservicios:

Ventajas	Inconvenientes
· Escalabilidad: cada servicio se puede escalar por separado. · Tamaño reducido, fácil administración · Implementación independiente · Facilidad de mantenimiento: los cambios son más localizados. · Despliegues autónomos: actualizaciones sin afectar a otros servicios. · Uso de tecnologías diversas: lenguaje o base de datos específica por servicio. · Resiliencia: el fallo de un servicio no implica la caída de toda la aplicación. · Despliegue sencillo y rápido, mediante el uso de contenedores.	· Complejidad operativa: más infraestructura y herramientas necesarias. Necesidad de comunicar los servicios entre sí. · Dificultad en pruebas integradas: validar el sistema completo es más complejo. · Problemas de consistencia de datos: requiere sincronización. · Sobrecarga de red: más tráfico entre servicios, así como mayor latencia. · Curva de aprendizaje inicial: especialmente para equipos no habituados. · Gestión distribuida de errores y logs: requiere herramientas específicas.

2.1.3 Comunicación entre microservicios

Para que una aplicación basada en microservicios funcione como un todo, sus servicios deben **comunicarse entre sí**. Esta comunicación puede ser de dos tipos:

- **Comunicación síncrona:** El servicio A solicita información al servicio B y espera su respuesta. Es simple de implementar y más intuitiva, pero si falla la comunicación el proceso se para. Generalmente se realiza mediante protocolos como **HTTP/REST** o **gRPC**.

  Ejemplo: el microservicio de "pedidos" consulta al microservicio de "usuarios" si un cliente está activo antes de confirmar una compra.

© Reservados todos los derechos de esta edición para POWEREDUCATION S.L., 2025

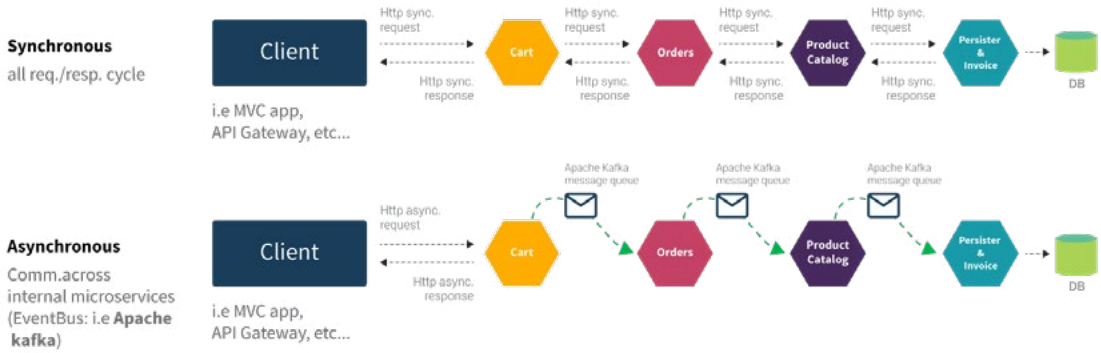


Ilustración 5. Diferencias entre comunicación síncrona y asíncrona en microservicios.

Fuente: https://devopedia.org/inter-service-communication-for-microservices

- **Comunicación asíncrona:** El servicio A **envía un mensaje a una cola o sistema de mensajería** (como RabbitMQ, Apache Kafka, que utilizan el protocolo AMQP – Adavanced Message Queuing Protocol), que se encargará de coordinar la entrega del mensaje al servicio B, por lo que A no espera respuesta inmediata. El servicio B procesará el mensaje cuando pueda, tomándolo de la cola o bus de eventos. Esta opción facilita el desacoplamiento, es más resiliente, aunque añade más complejidad.

  Ejemplo: cuando se confirma un pedido, se envía un evento a la cola "PedidosConfirmados", que será procesado por los servicios de "envíos" y "facturación".

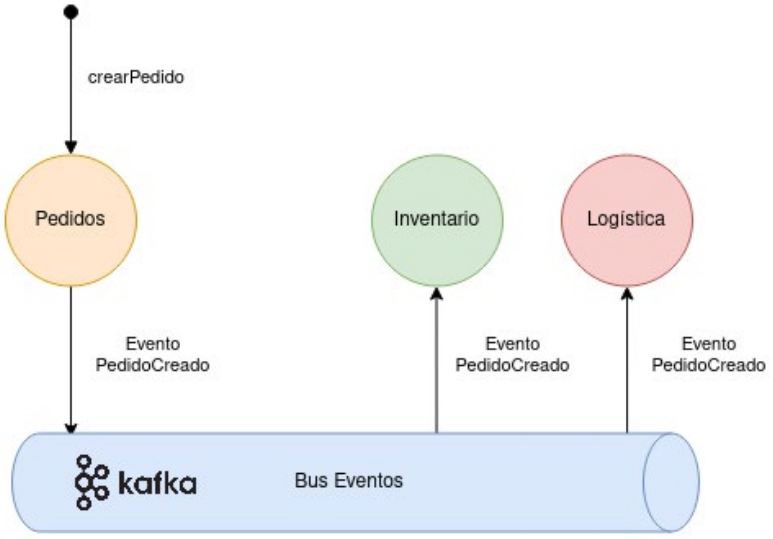
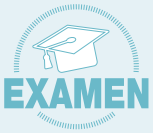


Ilustración 6. Ejemplo de comunicación asíncrona. Fuente: https://www.hiberus.com/crecemos-contigo/que-son-los-microservicios-basados-en-eventos-rompiendo-el-modelo-sincrono/

© Reservados todos los derechos de esta edición para POWEREDUCATION S.L., 2025

2.1.4 Integración entre microservicios: Orquestación y Coreografía

Cuando varios microservicios deben trabajar juntos para realizar una tarea, existen dos enfoques diferentes para coordinar la actividad entre ambos:



- **Orquestación:** Un **servicio central actúa como coordinador** del proceso, de forma similar al director de orquesta. Define el flujo completo de acciones, decidiendo el orden de ejecución y controlando errores. Permite centralizar la lógica y es fácil de supervisar y mantener, pero introduce un punto de fallo único.

- **Coreografía:** No existe un coordinador central. Cada servicio **reacciona a eventos generados por otros servicios**, en una especie de "baile" colaborativo. La lógica de negocio se distribuye. Es más difícil de desarrollar, seguir y depurar. Se basa en el intercambio de mensajes entre servicios. Se adapta mejor a la arquitectura de microservicios.

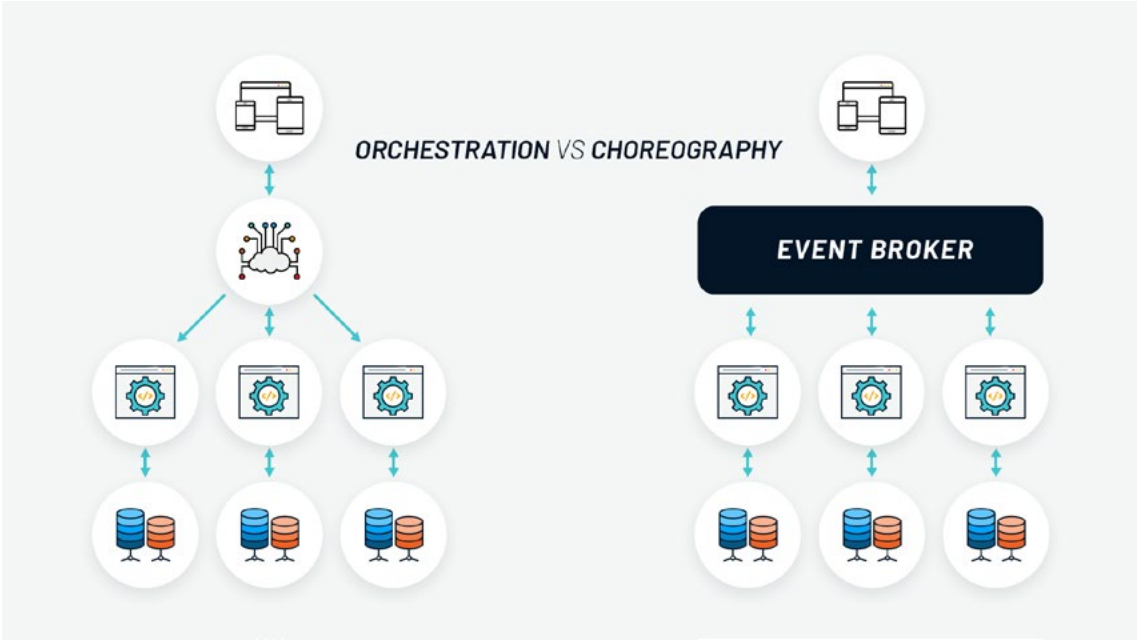


Ilustración 7. Comparativa entre orquestación y coreografía.

Fuente: https://blog.sparkfabrik.com/en/orchestration-vs-choreography

© Reservados todos los derechos de esta edición para POWEREDUCATION S.L., 2025

2.2 CONTENEDORES

Un **contenedor** es una unidad ejecutable de software que empaqueta una aplicación y todo su entorno de ejecución (bibliotecas, dependencias, archivos de configuración...) para garantizar que siempre se ejecuta de la misma forma, independientemente del sistema operativo o infraestructura subyacente. Los contenedores se caracterizan por causar una sobrecarga muy reducida. Se trata de una virtualización a nivel de sistema operativo.

Existen diferencias sustanciales entre un contenedor y una máquina virtual, que se recogen en la siguiente tabla:

Característica	Contenedor	Máquina virtual (VM)
SSOO	Comparte el kernel del host	Incluye su propio sistema operativo
Aislamiento	Aislado a nivel de procesos y red	Aislado completamente (hardware virtualizado)
Tamaño	Ligero (decenas o cientos de MB)	Pesado (varios GB)
Arranque	Muy rápido (segundos)	Más lento (minutos)
Rendimiento	Cercano al nativo	Menor rendimiento por sobrecarga de virtualización
Casos de uso ideales	Microservicios, CI/CD, despliegue rápido	Entornos que requieren total aislamiento o SO propio

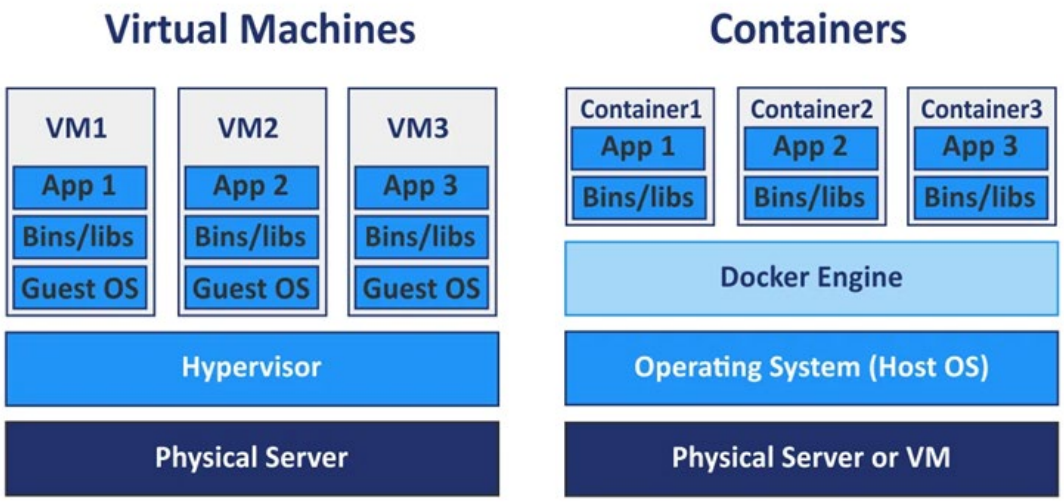


Ilustración 8. Diferencias entre VM y Contenedores. Fuente: https://www.nakivo.com/es/blog/docker-vs-kubernetes/

2.2.1 Arquitectura de contenedores

La arquitectura de contenedores se basa en varios componentes principales:

1. **Imagen (Image)**: Es una plantilla de solo lectura con todo lo necesario para ejecutar una aplicación (código, librerías, variables, etc.). Las imágenes se crean normalmente a partir de un Dockerfile.

2. **Contenedor (Container)**: Es una instancia en ejecución de una imagen. Puede tener su propio sistema de archivos, red y procesos.

© Reservados todos los derechos de esta edición para POWEREDUCATION S.L., 2025

3. **Motor de contenedores (Container Runtime)**: Software que gestiona imágenes y contenedores. El más conocido es **Docker Engine**, pero existen otros como **containerd**, **CRI-O** o **Podman**.

4. **Registro (Registry)**: Repositorio central donde se almacenan y publican las imágenes. Ejemplo: **Docker Hub**, **GitHub Container Registry** o **Harbor**.

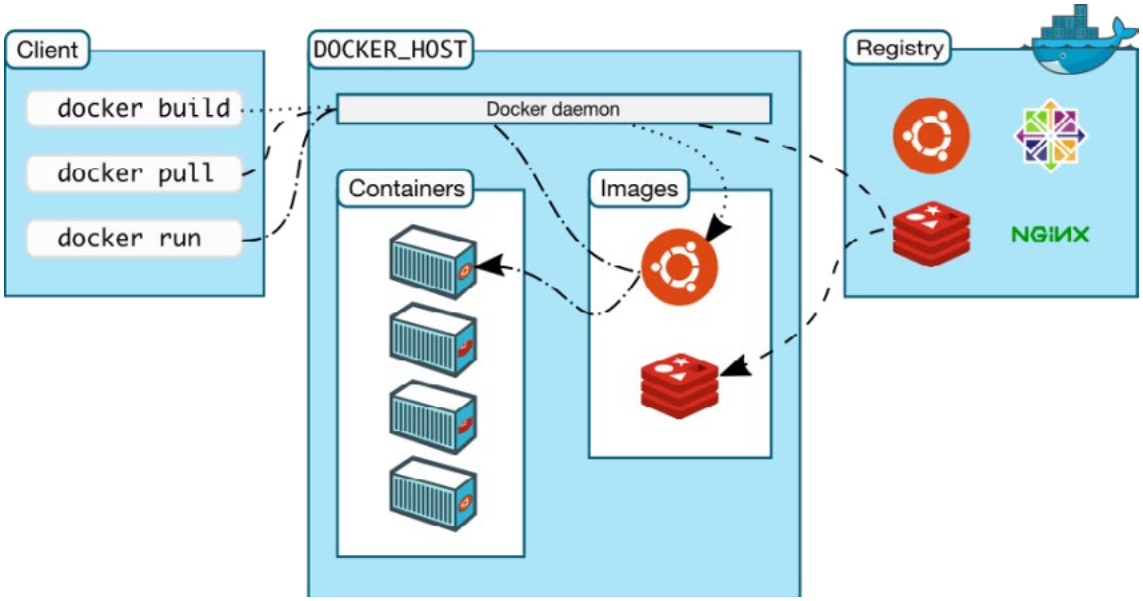


Ilustración 9. Arquitectura Docker.
Fuente: https://despliegue.codeandcoke.com/apuntes:docker

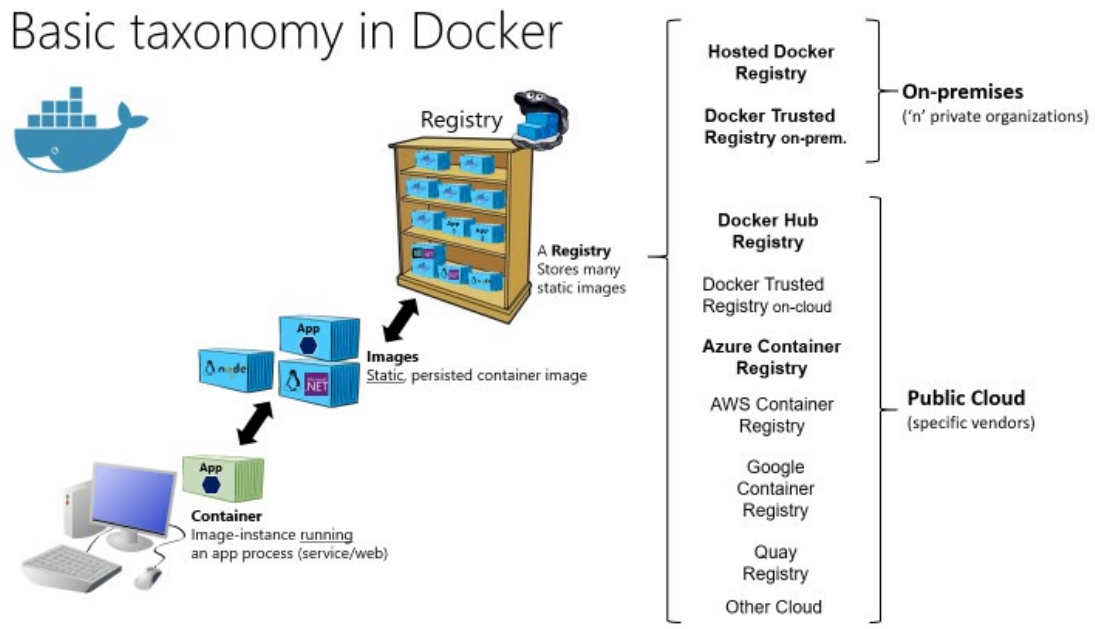


Ilustración 10. Taxonomía de contenedores en Docker.

Fuente: https://learn.microsoft.com/es-es/dotnet/architecture/microservices/container-docker-introduction/docker-containers-images-registries

© Reservados todos los derechos de esta edición para POWEREDUCATION S.L., 2025

2.2.2 Características. Ventajas e inconvenientes.

Ventajas	Inconvenientes
Uso eficiente de recursos, arranque rápido, rendimiento cercano al nativo.	Menor aislamiento que una VM. Un fallo en el kernel puede provocar la caída de todos.
Portabilidad entre entornos y sistemas	Requiere gestión adicional en producción
Simplifican la integración y entrega continua (CI/CD)	La persistencia de datos debe gestionarse por separado
Escalabilidad sencilla con herramientas de orquestación	Necesidad de herramientas complementarias para despliegue y seguridad

2.3 PLATAFORMAS DE CONTENEDORES Y MICROSERVICIOS



Estas plataformas facilitan la **gestión, despliegue y orquestación** de contenedores en entornos complejos o en producción, especialmente en aplicaciones basadas en microservicios.

2.3.1 Docker

- **Docker** es el **motor de contenedores más utilizado**, de código abierto.
- Permite construir, ejecutar y compartir contenedores de forma sencilla y automatizada.
- Usa imágenes que se definen mediante Dockerfile.
- Permite ejecutar contenedores con docker run y gestionarlos con docker-compose para varios servicios.
- Se usa tanto en desarrollo como en entornos de pruebas y producción a pequeña escala.

2.3.2 Kubernetes

- **Kubernetes** (K8s) es una **plataforma de orquestación de contenedores** de código abierto, desarrollada por Google.
- Automatiza despliegues, escalado, balanceo de carga, monitorización y recuperación de las aplicaciones en contenedores.

© Reservados todos los derechos de esta edición para POWEREDUCATION S.L., 2025



- Los **objetos de Kubernetes** son las unidades fundamentales que describen el estado deseado del sistema: qué aplicaciones se están ejecutando, qué recursos están disponibles y cómo deben comportarse. Cada objeto es una **representación declarativa** del estado del clúster.

Objeto	Campos principales	Descripción
Pod	apiVersion, kind, metadata, spec	Define contenedores que se ejecutan juntos. En spec se indican los contenedores, volúmenes, puertos, etc.
ReplicaSet	apiVersion, kind, metadata, spec	Asegura un número deseado de Pods. En spec se define el número de réplicas y la plantilla de Pod (template).
Deployment	apiVersion, kind, metadata, spec	Administra despliegues de aplicaciones y actualizaciones. spec contiene el número de réplicas y el template del Pod.
Service	apiVersion, kind, metadata, spec	Expone un conjunto de Pods. spec incluye el tipo (ClusterIP, NodePort, etc.), puertos y selector de Pods.
ConfigMap	apiVersion, kind, metadata, data	Almacena configuraciones externas en formato clave-valor.
Secret	apiVersion, kind, metadata, data	Similar a ConfigMap, pero cifrado/base64. Almacena contraseñas, tokens, certificados, etc.
Namespace	apiVersion, kind, metadata	Divide el clúster en espacios lógicos. Se usa para organizar recursos por proyectos, equipos o entornos.
Ingress	apiVersion, kind, metadata, spec	Gestiona el acceso HTTP/HTTPS desde el exterior al clúster. En spec se define el backend y las reglas de enrutamiento.

- Cada objeto puede tener cuatro campos principales. Estos campos forman la **estructura básica de casi todos los manifiestos de Kubernetes**, escritos normalmente en YAML o JSON. Comprenderlos es clave para definir correctamente cualquier recurso dentro del clúster.:



- **apiVersion**: Indica la versión de la API de Kubernetes que se usará para interpretar el objeto.
- **Kind**: Especifica el tipo de objeto que se está definiendo. Permite a Kubernetes saber cómo debe tratar el contenido del manifiesto.
- **Metadata**: Contiene información que identifica y organiza el objeto. Ejemplo de campos comunes son name, namespace, labels y annotations.
- **Spec**: Define el estado deseado del objeto. Su contenido varía según el tipo de objeto.

© Reservados todos los derechos de esta edición para POWEREDUCATION S.L., 2025

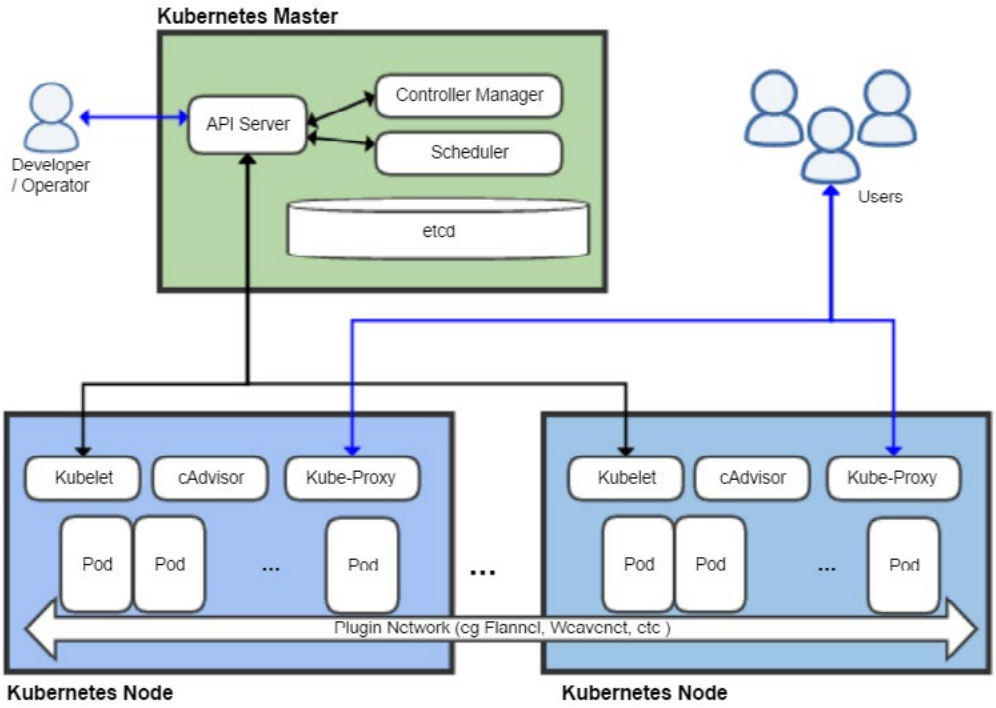


Ilustración 11. Arquitectura Kubernetes. Fuente: https://es.wikipedia.org/wiki/Kubernetes

2.3.3 OpenShift

- **OpenShift** es una **plataforma empresarial** PaaS basada en Kubernetes, mantenida por Red Hat. Utiliza orquestación de contenedores mediante Docker.
- Añade herramientas para desarrolladores (consolas web, CI/CD, seguridad, autenticación integrada).
- Incluye un catálogo de plantillas e integraciones con Jenkins, Git, S2I (source-to-image).
- Cuenta con gran robustez y soporte especializado de Red Hat.

© Reservados todos los derechos de esta edición para POWEREDUCATION S.L., 2025

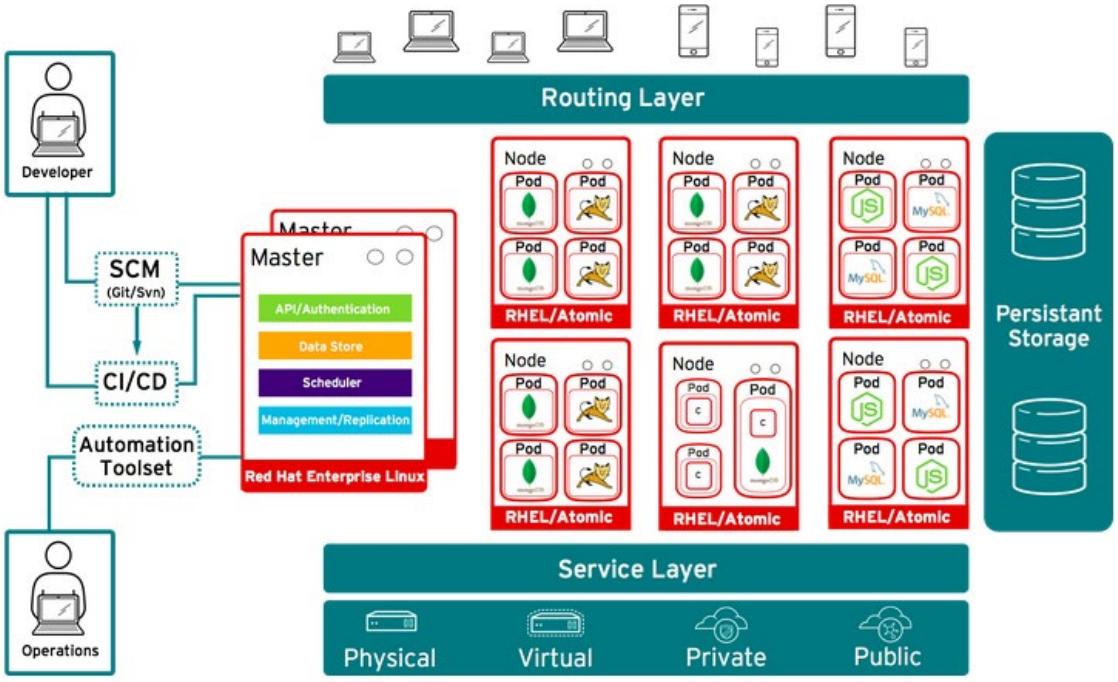


Ilustración 12. Arquitectura de Openshift.

Fuente: https://www.redhat.com/en/blog/openshift-enterprise-3-evolving-paas-future

2.3.4 Rancher

- **Rancher** es una solución **de gestión centralizada de clústeres Kubernetes**, ideal para entornos híbridos o multinube.
- Permite crear y controlar múltiples clústeres de K8s desde una única consola.
- Añade capas de seguridad, monitorización, control de acceso y soporte multiclúster.
- Compatible con Kubernetes nativo, EKS (Amazon), AKS (Azure), GKE (Google) y OpenShift.

2.3.5 Apache Mesos

- **Apache Mesos** es un **gestor de recursos distribuido** de código abierto, que permite ejecutar aplicaciones en entornos escalables.
- Aunque no está centrado exclusivamente en contenedores, permite su orquestación a gran escala.
- Se utiliza en combinación con Chronos, **Marathon** o **Aurora** para el despliegue de contenedores.
- Su uso ha disminuido en favor de Kubernetes, pero sigue siendo útil en entornos de alto rendimiento y Big Data.

© Reservados todos los derechos de esta edición para POWEREDUCATION S.L., 2025

2.3.6 Otras plataformas relevantes

Podman

- Alternativa a Docker, desarrollada por Red Hat.
- Compatible con imágenes Docker, pero no requiere daemon.
- Diseñado para entornos más seguros y rootless (sin privilegios de root).
- Usado principalmente en entornos Linux.

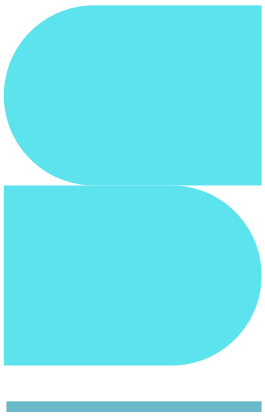
Nomad (de HashiCorp)

- Plataforma ligera para orquestar contenedores y aplicaciones no contenedorizadas.
- Muy sencilla de usar en comparación con Kubernetes.
- Se integra fácilmente con otras herramientas de HashiCorp como Vault y Consul.

Containerd

CRI-O

© Reservados todos los derechos de esta edición para POWEREDUCATION S.L., 2025



© Reservados todos los derechos de esta edición para POWEREDUCATION S.L., 2025.

Queda rigurosamente prohibida cualquier forma de reproducción, distribución comunicación pública o transformación total o parcial de esta obra sin el permiso escrito de los titulares de los derechos de explotación.